Progetto sistemi distribuiti e paralleli

Santoro Matteo

A.A 2024/25



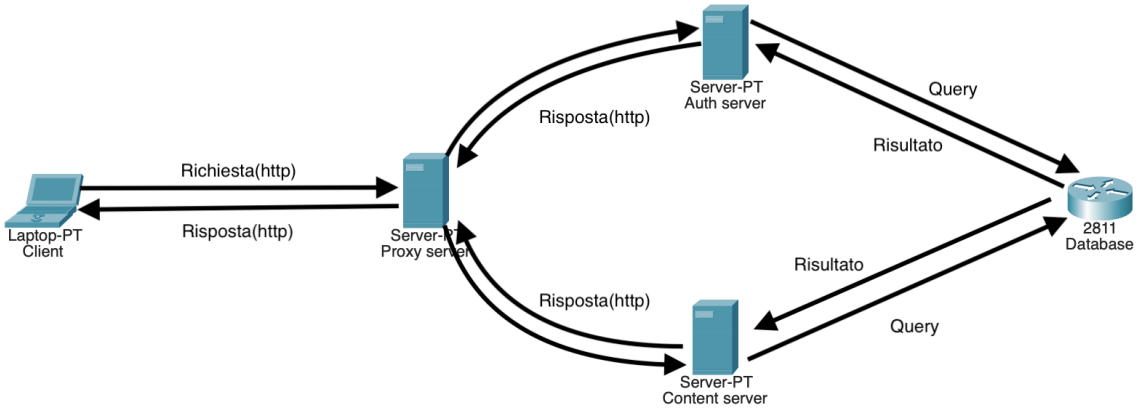
A.D. 1308

unipg

UNIVERSITÀ DEGLI STUDI
DI PERUGIA

GESTIONE SITO E-COMMERCE PER VENDITA DI PRODOTTI PER LA PALESTRA

Per questo progetto è stato realizzato un web service per gestire un sito e-commerce riguardante prodotti per la palestra. E' stato utilizzato Node.js per realizzare la parte backend del sito-e commerce,ovvero i server che gestiscono le richieste degli utenti come la visualizzazione dei prodotti, la registrazione, il login, gli ordini ecc… Inoltre è stato utilizzato Express per creare le API che comunicano con il frontend ed il database. Le comunicazione tra client e server vengono realizzate tramite il paradigma REST/JSON ed avvengono tramite fetch con metodi, GET, POST, PUT, DELETE ecc…



UTENTI E LE LORO FUNZIONALITA'

- Per un utente che non ha effettuato l'accesso è possibile visionare il sito insieme ai prodotti ma non può acquistarli, può contattare l'azienda tramite messaggio e non può lasciare recensioni ma può consultarle.
- Per un utente loggato invece è possibile acquistare i prodotti, lasciare recensioni fare ordini, aggiungere prodotti al carrello.
- L'utente admin invece ha dei permessi superiori e può gestire il sistema andando ad aggiungere, modificare e rimuovere prodotti, visualizzare tutti gli ordini effettuati dai clienti, visualizzare tutti gli utenti registrati ecc…

INFORMAZIONI MEMORIZZATE NEL DATABASE

Queste sono le tabelle utilizzate nel database palestra.

Tabella per la memorizzazione degli utenti

```sql
CREATE TABLE users (
  id INT AUTO_INCREMENT PRIMARY KEY,
  nome VARCHAR(100) NOT NULL,
  cognome VARCHAR(100) NOT NULL,
  email VARCHAR(255) NOT NULL UNIQUE,
  password VARCHAR(255) NOT NULL,
  is_admin BOOLEAN DEFAULT FALSE,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

Tabella per la memorizzazione dei prodotti

```sql
CREATE TABLE IF NOT EXISTS products (
  id INT AUTO_INCREMENT PRIMARY KEY,
  name VARCHAR(255) NOT NULL,
  description TEXT,
  image_url VARCHAR(255) NOT NULL,
  price DECIMAL(10,2) NOT NULL,
  quantity INT DEFAULT 0,
  category_id INT,
  discount_price DECIMAL (10,2) NOT NULL,
  FOREIGN KEY (category_id) REFERENCES categories(id)
);
```

Tabella per la memorizzazione dell categorie dei prodotti

```sql
CREATE TABLE IF NOT EXISTS categories (
  id INT AUTO_INCREMENT PRIMARY KEY,
  name VARCHAR(100) NOT NULL UNIQUE
);
```

Tabelle per gli ordini

```sql
CREATE TABLE IF NOT EXISTS orders (
  id INT AUTO_INCREMENT PRIMARY KEY,
  user_id INT NOT NULL,
  total_price DECIMAL(10,2) NOT NULL,
  shipping_address VARCHAR(255) NOT NULL,
  order_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  status ENUM('draft', 'confirmed') NOT NULL DEFAULT 'draft',
  FOREIGN KEY (user_id) REFERENCES users(id)
);


CREATE TABLE IF NOT EXISTS order_items (
  id INT AUTO_INCREMENT PRIMARY KEY,
  order_id INT NOT NULL,
  product_id INT NOT NULL,
  quantity INT NOT NULL,
  price DECIMAL(10,2) NOT NULL,
  FOREIGN KEY (order_id) REFERENCES orders(id),
  FOREIGN KEY (product_id) REFERENCES products(id)
);
```

Tabella per la memorizzazione delle recensioni

```sql
CREATE TABLE IF NOT EXISTS reviews (
  id INT AUTO_INCREMENT PRIMARY KEY,
  user_id INT NOT NULL,
  product_id INT NOT NULL,
  review_text TEXT NOT NULL,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (user_id) REFERENCES users(id),
  FOREIGN KEY (product_id) REFERENCES products(id)
  ON DELETE CASCADE
);
```

Tabella per la memorizzazione dei messaggi

```sql
CREATE TABLE IF NOT EXISTS messages (
  id INT AUTO_INCREMENT PRIMARY KEY,
  nome VARCHAR(50) NOT NULL,
  cognome VARCHAR(50) NOT NULL,
  email VARCHAR(100) NOT NULL,
  telefono VARCHAR(20),
  messaggio TEXT NOT NULL,
  accetta_terms BOOLEAN NOT NULL DEFAULT false,
  accetta_gdpr BOOLEAN NOT NULL DEFAULT false,
  data_invio TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

Tabella per la memorizzazione dei codici sconto

```sql
CREATE TABLE  IF NOT EXISTS discount_codes (
  id INT AUTO_INCREMENT PRIMARY KEY,
  code VARCHAR(50) NOT NULL UNIQUE,
  discount_percent DECIMAL(5,2) NOT NULL CHECK (discount_percent BETWEEN 0 AND 100),
  active BOOLEAN NOT NULL DEFAULT TRUE,
  expiration_date DATE NULL
);
```

GESTIONE SESSIONE UTENTE LOGGATO TRAMITE TOKEN

Per far si che alcune funzioni siano disponibili solo ad utenti loggati c'è bisogno di accertarsi che questi effettivamente lo siano. Ciò è possibile tramite il jsonwebtoken e i cookie. Quando un utente effettua il login con le credenziali adeguate viene realizzato un token che viene inviato al browser del client. Questo token contiene un payload con tutte le informazioni dell'utente che ha effettuato l'accesso, successivamente quest'ultimo viene firmato tramite JWT_SECRET ed infine inviato al browser del client. Ad ogni richiesta di autenticazione viene verificato il token tramite middleware,

Creazione token e impostazione dei cookie

```
const payload = {
  id: user.id,
  nome: user.nome,
  cognome: user.cognome,
  email: user.email,
  is_admin: user.is_admin,
  created_at: user.created_at,
};

const token = jwt.sign(payload, JWT_SECRET, {
  algorithm: "HS256",
  expiresIn: "1h",
});

res.cookie("token", token, {
  httpOnly: true,
  secure: true,
  maxAge: 3600000,
  sameSite: "Strict",
});
```

Middleware per la verifica della validità del token

```
// Middleware di autenticazione con JWT
function authenticateToken(req, res, next) {
  const token = req.cookies.token;

  if (!token) {
    if (req.originalUrl.startsWith("/api")) {
      return res.status(401).json({ success: false, message: "Token mancante o non valido" });
    }
    return res.redirect(302, "/login_register.html");
  }

  try {
    const payload = jwt.verify(token, JWT_SECRET);
    req.user = payload;
    next();
  } catch (err) {
    return res.redirect(302, "/index.html");
  }
}
```

Middleware per proteggere le funzionalità che ha solo l’admin

```
// Middleware per controllo admin
function admin(req, res, next) {
  if (req.user && req.user.is_admin) {
    next();
  } else {
    return res.status(403).send("Accesso negato: permessi insufficienti");
  }
}
```

SERVER

I server utilizzati per la gestione di questo sito e-commerce sono tre:

- Proxy server: questo server fa da “ponte” tra i client e i server, riceve tutte le richieste e le instrada nei server corrispondenti, si occupa anche di verificare i token e di verificare tramite middleware se l’utente sia admin o meno.
- Auth server: server che si occupa della gestione della registrazione e dell’accesso degli utenti.
- Content server: server che si occupa di tutte le possibili funzionalità che ogni utente loggato, non loggato o admin hanno.

Server proxy

```
const express = require("express");
const { createProxyMiddleware } = require("http-proxy-middleware");
const cookieParser = require("cookie-parser");
const { authenticateToken, admin } = require("./middlewares/middleware");

const app = express();
const port = 3000;

app.use(cookieParser())
const authTarget = "http://localhost:3001";
const contentTarget = "http://localhost:3002";

;

app.use(express.static("public"));

// Sezioni protette
app.use("/private", authenticateToken, express.static("private"));
app.use("/admin", authenticateToken, admin, express.static("admin"));

// Proxy per API autenticazione (login, registrazione, logout, userinfo)
app.use("/api/login", createProxyMiddleware({…
}));
app.use("/api/registr", createProxyMiddleware({…
}));
app.use("/api/logout", createProxyMiddleware({…
}));
app.use("/api/userinfo", createProxyMiddleware({…
}));

// Proxy per altre API (contenuti, prodotti ecc)
app.use("/api", createProxyMiddleware({…
}));


// Avvio server
app.listen(port, () => {…
});
```

Auth server

```
const express = require("express");
const mysql = require("mysql2");
const cookieParser = require("cookie-parser");
const jwt = require("jsonwebtoken");
const { authenticateToken, admin } = require("./middlewares/middleware");

const app = express();
const port = 3001;
const JWT_SECRET = "mia_chiave_super_segreta";

app.use(cookieParser());
app.use(express.json());

const connection = mysql.createPool({…
});

// LOGIN
app.post("/api/login", async (req, res) => {…
});


// REGISTRAZIONE
app.post("/api/registr", async (req, res) => {…
});

// INFO UTENTE (protetto)
app.get("/api/userinfo", authenticateToken, (req, res) => {…
});

// LOGOUT
app.post("/api/logout", authenticateToken, (req, res) => {…
});

app.listen(port, () => {…
});
```

Content server

```
const express = require("express");
const mysql = require("mysql2");
const cookieParser = require("cookie-parser");
const { authenticateToken, admin } = require("./middlewares/middleware");

const app = express();
const port = 3002;

app.use(express.json());
app.use(cookieParser());

// Connessione al database
const connection = mysql.createPool({…
});

// 🛒 Ordini dell'utente autenticato
app.get("/api/user/orders", authenticateToken, async (req, res) => {…
});

// 📦 Tutti i prodotti con nome categoria
app.get("/api/products", async (req, res) => {…
});



// ⭐ Prodotti in evidenza (primi 6)
app.get("/api/products/featured", async (req, res) => {…
});

// 🛰️ Prodotti in sconto
app.get("/api/products/discounted", async (req, res) => {…
});

// 🔍 Ricerca prodotti per nome
app.get("/api/products/search", async (req, res) => {…
});


app.post("/api/cart/add", authenticateToken, async (req, res) => {…
});
```

```
app.get("/api/cart", authenticateToken, async (req, res) => {…
});

app.put("/api/cart/update", authenticateToken, async (req, res) => {…
});

app.delete("/api/cart/remove", authenticateToken, async (req, res) => {…
});

app.post("/api/order/confirm", authenticateToken, async (req, res) => {…
});

app.get("/api/discount/validate", async (req, res) => {…
});

app.post("/api/contact", async (req, res) => {…
});



app.post("/api/reviews", authenticateToken, async (req, res) => {…
});

app.get("/api/reviews", async (req, res) => {…
});

app.get("/api/categories", async (req, res) => {…
});


// --- API ADMIN protette ---

// Visualizza tutti gli utenti
app.get("/api/admin/users", authenticateToken, admin, async (req, res) => {…
});

// Visualizza tutti gli ordini con info utente
app.get("/api/admin/orders", authenticateToken, admin, async (req, res) => {…
});
```

```
// Visualizza i dettagli di un ordine specifico
app.get("/api/admin/orders/:id", authenticateToken, admin, async (req, res) => {…
});

// Recupera tutti i prodotti (per dashboard admin)
app.get("/api/admin/products", authenticateToken, admin, async (req, res) => {…
});

// Aggiungi nuovo prodotto
app.post("/api/admin/products", authenticateToken, admin, async (req, res) => {…
});

// Modifica prodotto
app.put("/api/admin/products/:id", authenticateToken, admin, async (req, res) => {…
});

// Rimuovi prodotto
app.delete("/api/admin/products/:id", authenticateToken, admin, async (req, res) => {…
});

// Elimina utente
app.delete("/api/admin/users/:id", authenticateToken, admin, async (req, res) => {…
});

// Modifica is_admin di un utente
app.patch("/api/admin/users/:id", authenticateToken, admin, async (req, res) => {…
});

app.get("/api/admin/messaggi", authenticateToken, admin, async (req, res) => {…
});



//  Dettaglio prodotto per ID
app.get("/api/products/:id", async (req, res) => {…
});

// Avvio server
app.listen(port, () => {…
});
```